n8n Assignments — Solutions Guide

For each assignment, this guide repeats the task given to trainees, then explains the solution workflow node by node: what each node does, how it is configured, and how to test it.

#	Assignment	Solution file(s)	Nodes
1	Student Registration Form	`A1_Student_Registration.json`	4
2	Complaint Sorter	`A2_Complaint_Sorter.json`	7
3	Library Book Check	`A3_Library_Book_Check.json`	7
4	Currency Converter	`A4_Currency_Converter.json`	4
5	Course Enrolment with Welcome Email	`A5_Main_Course_Enrolment.json` + `A5_Sub_Welcome_And_Enrol.json`	5 + 3

Setup before testing

1. Run `00_assignments_setup.sql` once in the Supabase SQL Editor. It creates two new tables, `students` (Assignments 1 and 5) and `library_books` (Assignment 3, with 4 sample books). It never alters existing tables and is safe to run again.
2. Import the solution files. For Assignment 5, import the **sub-workflow first**, then the main workflow.
3. If a Gmail or Supabase node shows a red warning, open it and choose the account from the dropdown.
4. Test from Postman: method `POST`, Body, raw, JSON. Use the test URL `https://<your-n8n>/webhook-test/<path>` after clicking Listen for test event.

Every node in the solution files carries a one-line note on the canvas explaining what it does, and each workflow has a sticky note with its test data.

Assignment 1: Student Registration Form

The assignment

A college wants to collect student details through a form. Build a workflow that receives a student's name, email and course through a webhook, cleans the email (no spaces, lowercase), and saves the details in a database table. It should reply "Registration saved".

The solution

Flow: Webhook, then Edit Fields, then Supabase, then Respond to Webhook.

1. **Webhook — Student Registration** (Webhook). Method `POST`, path `assignment-1-register`. Response mode is "Using Respond to Webhook Node", so the caller waits for the final reply. The data sent arrives inside `body`.

2. **Edit — Clean Student Data** (Edit Fields). Builds three clean fields and drops everything else:
   - `name` = `($json.body.name || 'Unknown').trim()` : removes spaces; "Unknown" if missing.
   - `email` = `($json.body.email || '').toLowerCase().trim()` : `" ANITA.RAO@Example.com "` becomes `anita.rao@example.com` . The `|| ''` stops the workflow crashing when no email is sent.
   - `course` = `($json.body.course || 'Not given').trim()` .
3. **Supabase — Save Student** (Supabase, Create a Row). Table `students` ; columns `name` , `email` and `course` are mapped to the cleaned fields. The database fills `id` , `created_at` and `status` (`new`) by itself and returns the new row.
4. **Respond — Registration Saved** (Respond to Webhook). Replies with JSON: `message` "Registration saved", `student_id` and the cleaned `email` .

Testing

```
{ "name": "Anita Rao", "email": "  ANITA.RAO@Example.com ", "course": "Data Analytics" }
```

Reply: `{ "message": "Registration saved", "student_id": 1, "email": "anita.rao@example.com" }` , and a new row in `students` .

```
{ "name": "Rahul Menon", "course": "Web Design" }
```

Missing email: the workflow still completes and saves an empty email instead of crashing.

Key teaching point: clean data early. The `|| default` pattern is what makes the workflow survive missing input.

Assignment 2: Complaint Sorter

The assignment

A company receives complaints about delivery, payment and product. Build a workflow that reads the complaint type and sends each one to the right team: "Delivery Team", "Accounts Team" or "Product Team". Any other type goes to "General Team". It should reply with the team name.

The solution

Flow: Webhook, then Switch, which splits into four branches (Delivery, Payment, Product, Other), then all four join at one Respond to Webhook.

1. **Webhook — New Complaint** (Webhook). Method `POST` , path `assignment-2-complaint` , response mode "Using Respond to Webhook Node".
2. **Router — Complaint Type** (Switch, Rules mode). Checks `$json.body.complaint_type` , ignoring capitals:
   - Output "Delivery": equals `delivery` .
   - Output "Payment": equals `payment` .
   - Output "Product": equals `product` .

    - Fallback output "Other" (Options, Fallback Output = Extra Output): anything else, including a missing type. Nothing is lost.
3. **Edit — Delivery Team / Edit — Accounts Team / Edit — Product Team / Edit — General Team** (four Edit Fields nodes, one per branch). Each sets `team` to its team name and copies `customer` and `details` from the webhook. Only one branch runs per request.
4. **Respond — Team Assigned** (Respond to Webhook). All four branches connect into this one node. Replies with `assigned_to` (the team) and `customer`.

Testing

```
{ "complaint_type": "delivery", "customer": "Meera", "details": "Parcel arrived late" }
```

```
{ "complaint_type": "payment", "customer": "John", "details": "Charged twice" }
```

```
{ "complaint_type": "product", "customer": "Sara", "details": "Item was damaged" }
```

```
{ "complaint_type": "website", "customer": "Ali", "details": "Page will not load" }
```

Replies: Delivery Team, Accounts Team, Product Team, and General Team (fallback). In the editor, the highlighted path changes with each test.

Key teaching point: IF has two exits; Switch has as many as needed plus a fallback, so unexpected values still get handled.

Assignment 3: Library Book Check

The assignment

A library keeps its books in a table with the book name and whether it is available. Build a workflow that receives a book name and looks it up. If the book is available, email the student "Your book is available, come and collect it." If it is not found, email "Sorry, we don't have this book." Extra challenge: a book that exists but is borrowed gets "This book is currently borrowed."

The solution

Flow: Webhook, then Supabase lookup, then IF (found?). If found, a second IF (available?) chooses between the "available" and "borrowed" emails. If not found, the "not found" email is sent.

1. **Webhook — Book Request** (Webhook). Method `POST`, path `assignment-3-library`. Default response mode, so the caller gets "Workflow was started" straight away while the emails are sent in the background.
2. **Supabase — Find Book** (Supabase, Get Many). Table `library_books`, limit 1. Filter type "String", filter `book_name=ilike.<book name>`. `ilike` ignores capitals, so "atomic habits" still matches "Atomic Habits".

    - **Always Output Data is ON** (node Settings). Without it, a book that is not found produces no output and the workflow silently stops, so the "not found" email would never be sent.
3. **IF — Book Found?** (IF). Condition: `$json.id` is not empty. A found book has an `id` (true); the empty item does not (false).
4. **IF — Available?** (IF, true branch of step 3). Condition: `$json.available` is true.
5. **Gmail — Book Available** (Gmail, true branch of step 4). Subject "Your book is available: <book>"; body "Your book is available, come and collect it." with the title and author.
6. **Gmail — Book Borrowed** (Gmail, false branch of step 4). "This book is currently borrowed." This is the extra challenge.
7. **Gmail — Book Not Found** (Gmail, false branch of step 3). "Sorry, we don't have this book."

All three Gmail nodes send to `$('Webhook – Book Request').item.json.body.student_email`. They reach back to the webhook because the Supabase and IF nodes in between replaced `$json` with book data.

Testing

Use a real inbox in place of the sample address.

```
{ "book_name": "Atomic Habits", "student_email": "your-email@example.com" }
```

```
{ "book_name": "Wings of Fire", "student_email": "your-email@example.com" }
```

```
{ "book_name": "Harry Potter", "student_email": "your-email@example.com" }
```

Results: "available" email, "currently borrowed" email, "don't have this book" email.

Key teaching points: Always Output Data turns "nothing found" into a normal path; two IF nodes in a row give three outcomes; `$('Node Name')` reaches data from earlier nodes.

Assignment 4: Currency Converter

The assignment

A travel agency wants to show prices in different currencies. Build a workflow that receives an amount and a currency, calls a free exchange-rate API, calculates the converted amount, and replies with a short result such as "100 GBP = 125.4 USD".

The solution

Flow: Webhook, then HTTP Request, then Code, then Respond to Webhook.

1. **Webhook — Convert Request** (Webhook). Method `POST`, path `assignment-4-currency`, response mode "Using Respond to Webhook Node". Receives `amount`, `from` and `to`.
2. **HTTP — Get Exchange Rates** (HTTP Request). This is the API integration.
    - Method `GET`, URL `https://open.er-api.com/v6/latest/{{ ($json.body.from || 'GBP').toUpperCase().trim() }}`.

- Free API, no account or key, so no credential is needed.
- It returns `result: "success"` and a list of `rates` from the `from` currency to every other currency (for example `"USD": 1.32`). An unknown `from` code returns `result: "error"`.
- **Never Error is ON** (Options, Response). An error reply is passed on instead of stopping the workflow, so the Code node can turn it into a friendly message.

3. **Code — Calculate Conversion** (Code, JavaScript). Reads the original request with `$('Webhook – Convert Request').first().json.body` and the API reply with `$input.first().json`, then:
   - If the API result is not "success", returns `Unknown currency: <from>`.
   - If the amount is missing, zero or not a number, returns `Amount must be a number above 0`.
   - If the `to` currency is not in the rates list, returns `Unknown currency: <to>`.
   - Otherwise multiplies amount by rate, rounds to 2 decimals, and returns `ok: true`, `result` (for example "100 GBP = 132.31 USD"), the `rate` and the date of the rates.
4. **Respond — Converted Amount** (Respond to Webhook). Returns whatever the Code node produced: the result, or the clear error.

Testing

```
{ "amount": 100, "from": "GBP", "to": "USD" }
```

```
{ "amount": 2500, "from": "INR", "to": "EUR" }
```

```
{ "amount": 50, "from": "GBP", "to": "XYZ" }
```

Results: two conversions (rates change daily, so numbers vary) and `{ "ok": false, "error": "Unknown currency: XYZ" }` for the last test.

Key teaching points: HTTP Request connects to any API; Never Error plus a check in the Code node means bad input gives a clear answer, not a crash.

Assignment 5: Course Enrolment with Welcome Email

The assignment

A training institute wants a complete enrolment process. Build two workflows: a **main workflow** that receives name, email and course, saves the student with status "new" and calls the second workflow; and a **sub-workflow** that sends a welcome email and changes the status to "enrolled". The main workflow replies "Enrolled successfully". Bonus: a different welcome email per course.

The solution

This is the only assignment that uses a sub-workflow.

Main flow: Webhook, then Edit Fields, then Supabase insert, then **Run Sub** (calls the sub-workflow), then Respond to Webhook.

Sub flow: Start (called by main), then Gmail, then Supabase update. The sub-workflow's last output returns to the main workflow.

Main workflow: Assignment 5 — Course Enrolment (Main)

1. **Webhook — Enrolment** (Webhook). Method `POST`, path `assignment-5-enrol`, response mode "Using Respond to Webhook Node".
2. **Edit — Clean Enrolment** (Edit Fields). Cleans `name` (default "Student"), `email` (trimmed, lowercase) and `course` (default "General").
3. **Supabase — Save as New** (Supabase, Create a Row). Inserts into `students` with `status` = `new`. Returns the new row with its `id`, which travels to the sub-workflow.
4. **Run Sub — Welcome & Enrol** (Execute Workflow). **Calls the sub-workflow "Assignment 5 — Sub: Welcome Email & Enrol".** Mode "Run once for each item", Wait for Sub-Workflow Completion ON. It sends the saved row to the sub-workflow and waits for the updated row to come back. After import, open this node and select the sub-workflow from the list.
5. **Respond — Enrolled** (Respond to Webhook). Replies `message` "Enrolled successfully", `student_id` and `status` (now `enrolled`).

Sub-workflow: Assignment 5 — Sub: Welcome Email & Enrol

1. **Start — From Main** (Execute Workflow Trigger, accept all data). Runs only when the main workflow's **Run Sub — Welcome & Enrol** node calls it. It receives the saved student row.
2. **Gmail — Welcome Student** (Gmail). Sends to `$json.email`. Subject "Welcome to <course>!"; the body greets the student, names the course and gives the enrolment number `STU-<id>`. Because the course name is in the subject and body, each course gets its own email (the bonus).
3. **Supabase — Mark Enrolled** (Supabase, Update). Table `students`, filter `id` equals `$('Start – From Main').item.json.id`, sets `status` to `enrolled`. It updates the **same row** the main workflow created; it does not add a new one. It reaches back to the Start node for the id because Gmail replaced `$json`. Its output (the updated row) is what the main workflow receives.

Import order

1. Import `A5_Sub_Welcome_And_Enrol.json` and save it.
2. Import `A5_Main_Course_Enrolment.json`.
3. In the main workflow, open **Run Sub — Welcome & Enrol** and select "Assignment 5 — Sub: Welcome Email & Enrol".
4. Publish both.

Testing

```
{ "name": "Divya Nair", "email": "your-email@example.com", "course": "Python Basics" }
```

```
{ "name": "Kiran Das", "email": "your-email@example.com", "course": "Digital Marketing" }
```

Expected: reply "Enrolled successfully", a welcome email for that course, and the row in `students` changing from `new` to `enrolled` with the same `id`.

Key teaching points: a sub-workflow is a separate workflow started by an Execute Workflow node; data goes in through its Start node and comes back from its last node; saving first and updating later by `id` keeps one record per student.

Common problems

Symptom	Cause	Fix
`404 webhook is not registered`	Test URL used without clicking Listen for test event, or workflow not published	Click Listen first, or publish
Fields arrive empty	Body sent as Text or in Params	Postman: Body, raw, JSON
Red credential warning	Credential not on this n8n	Select the Gmail or Supabase account in the node
Assignment 3 stops after the lookup	Always Output Data is off	Turn it on in the Supabase node's Settings
Assignment 5: "workflow not found"	Sub imported after main, or not selected	Import the sub first, then select it in Run Sub
Supabase errors or timeouts	Free project paused	Restore it in the Supabase dashboard